

Diffie-Hellman y Criptografía de Clave Pública

Criptografía y Seguridad – 72.44 ITBA

Guía de estudio para examen

Índice

1. Protocolo de intercambio de claves	2
1.1. Experimento KE (ataques pasivos)	2
2. Protocolo Diffie-Hellman	2
2.1. Descripción paso a paso	2
2.2. Diagrama de intercambio	3
2.3. Por qué es difícil: problema del logaritmo discreto (DLP)	3
2.4. Ataque Man-in-the-Middle (MitM)	3
3. Criptosistema asimétrico	4
3.1. Prueba de indistinguibilidad	4
4. RSA (Rivest, Shamir, Adleman)	4
4.1. Generación de claves	4
4.2. Cifrado y descifrado	5
4.3. Por qué funciona: Teorema de Euler	5
4.4. Diagrama de generación, cifrado y descifrado	5
4.5. Problemas del textbook RSA	5
4.6. PKCS #1 v1.5: padding aleatorio	5
4.7. Tamaño de claves y uso práctico	6
5. ElGamal (basado en DH)	6
5.1. Curvas elípticas (ECC)	6
6. Firmas digitales	7
6.1. Ventajas sobre los MACs	7
6.2. Experimento Sig-Forge (infalsificabilidad)	7
6.3. RSA-Signature básico y sus ataques	7
6.4. Hashed RSA: la firma que se usa en la práctica	8
6.5. DSS / ECDSA	8
7. PKI y certificados digitales	8
7.1. Cadena de confianza	8
8. TLS handshake (resumen)	9
9. Tabla comparativa: MAC vs Firma digital	9
10. Ganchos mnemónicos	9
11. Ejercicios tipo examen con respuesta	10

1. Protocolo de intercambio de claves

Definición: Protocolo de intercambio de claves

Protocolo $\Pi(n)$ ejecutado por dos partes, sin entrada salvo el parámetro de seguridad n . Su salida es:

- Una **transcripción** (todos los mensajes intercambiados; el atacante pasivo la conoce).
- Una clave k_a que deriva la primera parte y k_b que deriva la segunda, cada una solo con la información de su propio extremo.

Condición fundamental: $k_a = k_b$ (la misma clave, derivada de forma independiente por ambas partes).

1.1. Experimento KE (ataques pasivos)

El experimento $\text{KE}_{A,\Pi}$ comprueba que el atacante no obtiene *ninguna* información sobre la clave negociada:

1. Se ejecuta Π ; sea $k = k_a = k_b$ la clave resultante.
 2. Se genera $b \leftarrow \{0, 1\}$ uniformemente.
 3. Si $b = 0$: se toma $k' \leftarrow \{0, 1\}^n$ (completamente aleatoria); si $b = 1$: $k' = k$ (la clave real).
 4. El atacante A recibe la transcripción Trans y k' , y emite $b' \in \{0, 1\}$.
- $\text{KE}_{A,\Pi} = 1$ si $b = b'$. Si $\Pr[\text{KE}_{A,\Pi} = 1] < \frac{1}{2} + \varepsilon$, el protocolo Π es seguro contra ataques pasivos.

Destacado

No se le pide al atacante *adivinar* la clave, sino *distinguir* la clave real de una aleatoria. Adivinar al azar o sesgar levemente da $\approx 0,5$; si llega a $\sim 0,6$ es que encontró información parcial y el protocolo falla.

2. Protocolo Diffie-Hellman

2.1. Descripción paso a paso

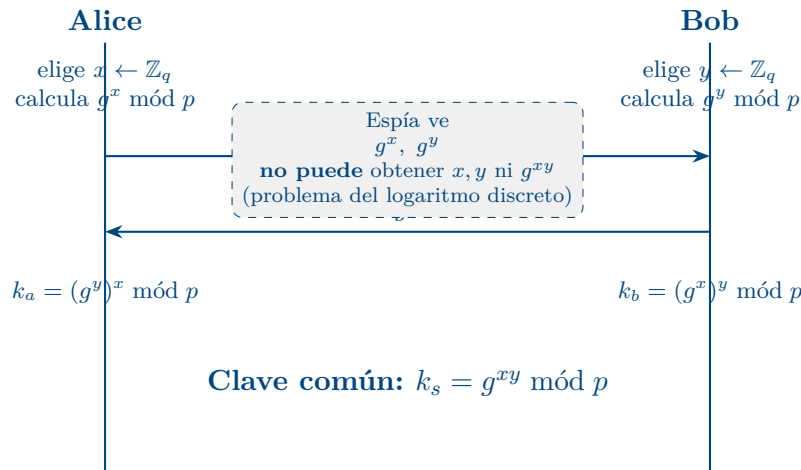
1. Alice define (G, q, g) : G grupo cíclico, q su orden (**primo**), g un generador. Esta información es **pública** (parte de la transcripción; el espía la conoce).
2. Alice elige $x \leftarrow \mathbb{Z}_q$ en secreto y calcula $h_1 = g^x \text{ mód } p$.
3. Alice envía a Bob: $(G, q, g, h_1 = g^x)$.
4. Bob elige $y \leftarrow \mathbb{Z}_q$ en secreto y calcula $h_2 = g^y \text{ mód } p$.
5. Bob envía a Alice: $h_2 = g^y$.
6. Alice calcula $k_a = h_2^x = (g^y)^x \text{ mód } p$.
7. Bob calcula $k_b = h_1^y = (g^x)^y \text{ mód } p$.

La clave de sesión común es:

$$k_s = (g^x)^y \text{ mód } p = (g^y)^x \text{ mód } p = g^{xy} \text{ mód } p,$$

pues la exponenciación sucesiva es conmutativa. Lo que viaja por el canal es g^x y g^y ; x **nunca sale de Alice** y y **nunca sale de Bob**.

2.2. Diagrama de intercambio



Diffie-Hellman: Alice y Bob intercambian g^x y g^y y derivan independientemente la clave g^{xy} . El espía ve los valores públicos pero no puede recuperar x , y ni g^{xy} (DLP).

2.3. Por qué es difícil: problema del logaritmo discreto (DLP)

En campos *infinitos* calcular logaritmos es eficiente (logaritmo natural, series de Taylor). En campos *finitos* (módulo p), el **efecto de envolvente modular** rompe esa estructura: $3^2 \text{ mód } 7 = 2$ (no 9). Dado $g^x \text{ mód } p$, recuperar el exponente x es el **problema del logaritmo discreto**, abierto por más de un siglo sin solución eficiente conocida.

Para la seguridad de DH se necesita además la **conjetura de decisión DH (DDH)**: dados g, g^x, g^y , un adversario no puede distinguir g^{xy} de un valor aleatorio.

[Examen]

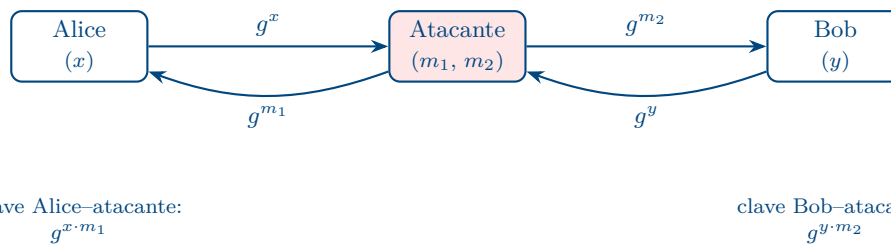
Preguntas típicas de parcial sobre DH:

- **Detallar el protocolo:** parámetros (G, q, g) , Alice elige $x \rightarrow g^x$, Bob elige $y \rightarrow g^y$, clave común g^{xy} .
- **V/F “DH permite enviar una clave a Bob”** → **FALSO**. DH acuerda/deriva una clave compartida; **no transporta** una clave preexistente. (2018, 2023.) No confundir *acuerdo de clave* con *transporte de clave*.
- **Seguridad = problema del logaritmo discreto.** Dado g^x y g^y no se puede obtener x , y ni g^{xy} .
- **¿Por qué q debe ser primo?** Se trabaja con grupos multiplicativos; q primo garantiza que todos los elementos del grupo son coprimos entre sí, evitando subgrupos con estructura factorizable. (Recup 2025, 2025-1.)

2.4. Ataque Man-in-the-Middle (MiTM)

¡OJO!

DH puro no tiene autenticación. Solo es seguro frente a atacantes **pasivos**. Un atacante **activo** que intercepta y modifica mensajes en tránsito realiza un ataque **Man-in-the-Middle (MiTM)**: negocia claves separadas con Alice y con Bob, y puede descifrar y reenviar todo el tráfico.



MiTM sobre DH puro (sin autenticación): el atacante sustituye los valores g^x y g^y y negocia claves separadas con cada parte. Alice y Bob creen hablar entre sí pero todo pasa por el atacante. Solución: complementar DH con MACs o firmas digitales.

3. Criptosistema asimétrico

Definición: Criptosistema asimétrico

Terna de algoritmos eficientes:

$\text{Gen} : () \rightarrow PK \times SK$ (genera **par** de claves pública/privada)
 $\text{Enc} : PK \times \mathcal{M} \rightarrow \mathcal{C}$ (cifra con la **pública**)
 $\text{Dec} : SK \times \mathcal{C} \rightarrow \mathcal{M}$ (descifra con la **privada**)

Propiedad básica: $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$ para todo m y todo par (pk, sk) válido.

3.1. Prueba de indistinguibilidad

Igual que en el mundo simétrico, pero se le entrega la clave **pública** al adversario:

1. $(pk, sk) \leftarrow \text{Gen}$.
2. El adversario A recibe pk y genera (m_0, m_1) .
3. $b \leftarrow \{0, 1\}$; se le entrega $c = \text{Enc}_{pk}(m_b)$.
4. A emite b' ; gana si $b = b'$.

Destacado

Como A conoce pk , puede cifrar cualquier mensaje *él mismo*: tiene acceso gratuito al oráculo de cifrado. Por eso:

$$\text{indistinguible (pasivo)} \Leftrightarrow \text{CPA-Secure}$$

Y como CPA-Secure **requiere no determinismo**, **no existen criptosistemas asimétricos seguros que sean determinísticos**.

4. RSA (Rivest, Shamir, Adleman)

4.1. Generación de claves

Elegir p, q primos fuertes grandes, $n = p \cdot q$.

$$\Phi(n) = (p-1)(q-1).$$

Elegir $e \in (1, \Phi(n))$ con $\text{gcd}(e, \Phi(n)) = 1$.

Calcular $d = e^{-1} \bmod \Phi(n)$ (Euclides extendido).

$$pk = (n, e), \quad sk = (n, d).$$

4.2. Cifrado y descifrado

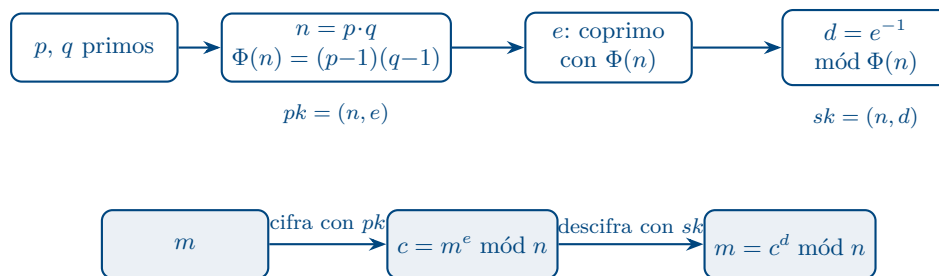
$$\text{Enc}_{pk}(m) = m^e \bmod n, \quad \text{Dec}_{sk}(c) = c^d \bmod n.$$

Los exponentes se calculan con **exponenciación rápida** (cuadrado y multiplica), reduciendo módulo n en cada paso para no manejar números de tamaño m^e .

4.3. Por qué funciona: Teorema de Euler

$$\begin{aligned} \text{Dec}(\text{Enc}(m)) &= (m^e)^d \bmod n = m^{ed} \bmod n. \\ e \cdot d &\equiv 1 \bmod \Phi(n) \Rightarrow e \cdot d = c \cdot \Phi(n) + 1 \text{ (para algún entero } c). \\ m^{ed} &= m^{c\Phi(n)+1} = (m^{\Phi(n)})^c \cdot m \bmod n. \\ \text{Euler: } m^{\Phi(n)} &\equiv 1 \bmod n \Rightarrow m^{ed} \equiv 1^c \cdot m = m \bmod n. \end{aligned}$$

4.4. Diagrama de generación, cifrado y descifrado



Arriba: generación de claves RSA. Abajo: cifrado con la clave pública (n, e) y descifrado con la clave privada (n, d) .

4.5. Problemas del textbook RSA

- **Determinístico:** mismo mensaje siempre produce el mismo cifrado $\Rightarrow$ no pasa la prueba de indistinguibilidad (no es CPA-Secure).
- **e y m chicos:** si $m^e < n$, la reducción modular no actúa y $c = m^e$ exactamente; basta con calcular la raíz e -ésima real. Históricamente $e = 3$ era popular.
- **Módulos repetidos:** si dos pares de claves comparten el mismo n , un atacante puede factorizarlo y recuperar las claves privadas.
- Los primos p, q deben ser “**primos fuertes**” (no cualquier primo).

4.6. PKCS #1 v1.5: padding aleatorio

Estándar que hace RSA no determinístico. Sea k la longitud de n en bytes; solo se cifran mensajes de hasta $k - 11$ bytes. Estructura del mensaje con padding:

$$m' = \underbrace{00000000}_{\text{byte 0}} \parallel \underbrace{00000010}_{\text{byte 2}} \parallel \underbrace{\overset{r}{\geq 8 \text{ bytes aleatorios, } r_i \neq 0}}_{\geq 8 \text{ bytes aleatorios, } r_i \neq 0} \parallel \underbrace{00000000}_{\text{separador}} \parallel m.$$

Al menos 8 bytes aleatorios $\Rightarrow \geq 2^{64}$ variantes por mensaje. Al descifrar se obtiene m' y se quita el padding buscando el byte 00.

[Examen]

RSA con padding aleatorio (PKCS #1 v1.5) se cree **CPA-Secure**, pero **no es CCA-Secure**: se encontraron ataques de texto cifrado elegido (el cifrado es manipulable). (2025-2.) El padding aleatorio resuelve CPA, no CCA.

4.7. Tamaño de claves y uso práctico

La seguridad de RSA es relativa al tamaño de $n = pq$. Como factorizar y el logaritmo discreto son *sub-exponenciales* (más rápidos que fuerza bruta), se necesitan claves grandes: hoy se recomiendan ≥ 2048 bits para n . RSA es 1–2 órdenes de magnitud **más lento que AES** y solo cifra mensajes de tamaño acotado por n , por eso en la práctica se usa para cifrar una **clave simétrica**, que luego cifra el mensaje largo.

[Examen]

RSA: preguntas de parcial centrales.

- Generación: $n = pq$, $\Phi(n) = (p-1)(q-1)$, e coprimo con $\Phi(n)$, $d = e^{-1} \bmod \Phi(n)$.
- Cifrado: $c = m^e \bmod n$. Descifrado: $m = c^d \bmod n$.
- Por qué funciona: Euler $m^{\Phi(n)} \equiv 1 \bmod n$.

5. ElGamal (basado en DH)

ElGamal es un Diffie-Hellman “desacoplado en el tiempo” convertido en criptosistema.

Gen: Elegir G, q, g ; $x \leftarrow \mathbb{Z}_q$; $h = g^x$.

$pk = (G, q, g, h)$, $sk = (G, q, g, x)$.

Enc_{pk}(m): $y \leftarrow \mathbb{Z}_q$; $c = (c_1, c_2) = (g^y, h^y \cdot m)$.

Dec_{sk}(c): $m = c_2 / c_1^x$.

$c_1 = g^y$ es la “segunda parte de DH”; $h^y = (g^x)^y = g^{xy}$ enmascara a m como un one-time pad. Al descifrar, $c_1^x = g^{yx}$ cancela la máscara.

Destacado

Por qué es CPA-Secure: si el problema de decisión DH (DDH) es difícil en G , entonces ElGamal es CPA-Secure. ElGamal es probabilístico por naturaleza (elige y al azar en cada cifrado) sin necesitar padding externo.

5.1. Curvas elípticas (ECC)

ElGamal funciona sobre cualquier grupo donde DDH sea difícil, incluyendo grupos de **curvas elípticas**. Ventaja clave:

Algoritmo	Seguridad (bits)	Tamaño de clave
RSA-2048	≈ 112	2048 bits
ECC-256	≈ 128	256 bits
ECC-320	≈ 160	320 bits

Misma seguridad con claves mucho más chicas $\Rightarrow$ operaciones más rápidas, certificados más compactos. Todo algoritmo de clave pública prefijado con “EC” suele ser ElGamal o DSA sobre una curva elíptica (ECDH, ECDSA).

6. Firmas digitales

Definición: Firma digital

Terna de algoritmos:

$$\text{Gen} : () \rightarrow PK \times SK$$

$$\text{Sign} : SK \times \mathcal{M} \rightarrow \mathcal{S} \quad (\text{firma con la } \mathbf{privada})$$

$$\text{Vrfy} : PK \times \mathcal{M} \times \mathcal{S} \rightarrow \{0, 1\} \quad (\text{verifica con la } \mathbf{pública})$$

Propiedad básica: $\text{Vrfy}_{pk}(m, \text{Sign}_{sk}(m)) = 1$.

Destacado

El rol de las claves está **al revés** que en el cifrado:

Operación	Clave usada	Objetivo
Cifrar	pública del destinatario	confidencialidad
Descifrar	privada del destinatario	(recuperar el mensaje)
Firmar	privada del emisor	integridad / autenticación
Verificar	pública del emisor	cualquiera puede verificar

6.1. Ventajas sobre los MACs

- **Públicamente verificables:** cualquier parte que tenga la clave pública puede verificar la firma. Un MAC necesita la clave secreta para verificar.
- **Transferibles:** la firma puede reenviarse a múltiples destinatarios o por una cadena; cada eslabón puede verificar.
- **No repudio:** quien firmó no puede negar haberlo hecho. La única forma de generar la firma es con la clave privada (nunca compartida). En Argentina hay ley de firma digital; un organismo del Estado valida que el dueño de la clave privada es tal persona real.

6.2. Experimento Sig-Forge (infalsificabilidad)

Análogo a Mac-Forge, pero el adversario también conoce pk :

1. $(sk, pk) \leftarrow \text{Gen}$.
2. El adversario A recibe pk y acceso al oráculo de firma $f(x) = \text{Sign}_{sk}(x)$.
3. A realiza las consultas que quiera (conjunto Q).
4. A emite (m, s) con $m \notin Q$.

$\text{Sig-forge}_{A, \Pi} = 1$ si $\text{Vrfy}_{pk}(m, s) = 1$ y $m \notin Q$. Si ningún adversario eficiente gana con probabilidad no despreciable, la firma es **existencialmente infalsificable (EUF-CMA)**.

6.3. RSA-Signature básico y sus ataques

Idéntico a RSA-Encryption pero invirtiendo los roles de las claves:

$$\text{Sign}_{sk}(m) = m^d \text{ mód } n, \quad \text{Vrfy}_{pk}(m, s) : s^e \equiv m \text{ (mód } n).$$

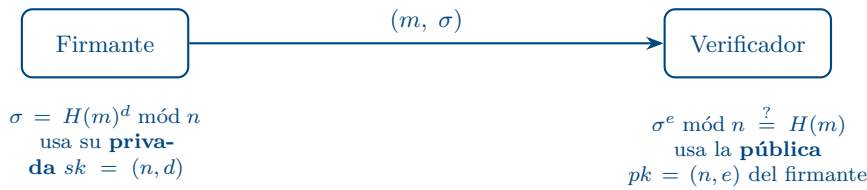
¡OJO!

RSA-Signature básico (sin hash) es INSEGURO. Dos ataques de falsificación:

- Elegir $s \leftarrow S$ al azar, calcular $m = s^e \text{ mód } n$, publicar (m, s) : **pasa la verificación**.
- **Maleabilidad:** dados (m_1, s_1) y (m_2, s_2) válidos, el par $(m_1 \cdot m_2, s_1 \cdot s_2)$ también verifica (*RSA* es multiplicativamente homomórfico).

6.4. Hashed RSA: la firma que se usa en la práctica

$$\text{Sign}_{sk}(m) = H(m)^d \text{ mód } n, \quad \text{Verfy}_{pk}(m, \sigma) : \sigma^e \stackrel{?}{=} H(m) \text{ (mód } n).$$



Hashed RSA: el firmante aplica hash y firma con su privada; el verificador rehash el mensaje recibido y compara con $\sigma^e \text{ mód } n$.

[Examen]

¿Por qué se firma el **HASH** y no el mensaje directo? (Parcial 2015)

- **Resistencia a preimagen** del hash evita el primer ataque: no se puede partir de s elegido y construir un $m = H^{-1}(s^e)$ válido.
- **No predicibilidad de $H(m)$** evita el ataque de maleabilidad.
- **Bonus:** el hash tiene tamaño fijo (e.g. 256 bits), así la firma es de tamaño fijo independientemente del tamaño del mensaje; RSA básico solo puede firmar mensajes menores que n .

Hashed RSA no tiene prueba de seguridad estándar; se demuestra seguros en el *modelo de oráculo aleatorio* (ROM).

6.5. DSS / ECDSA

Algoritmo de firma basado en ElGamal. Genera claves $(p, q, g, y)/(p, q, g, x)$ con q primo, p primo tal que $q \mid p-1$, g generador de orden q en $\mathbb{Z}_p^*$, $x \leftarrow \mathbb{Z}_q$, $y = g^x \text{ mód } p$.

Firma (r, s) :

$$r = (g^k \text{ mód } p) \text{ mód } q, \quad s = [H(m) + x \cdot r] \cdot k^{-1} \text{ mód } q.$$

ECDSA es la variante sobre curvas elípticas: firmas más compactas con claves más chicas, ampliamente usado en TLS, SSH y certificados modernos.

7. PKI y certificados digitales

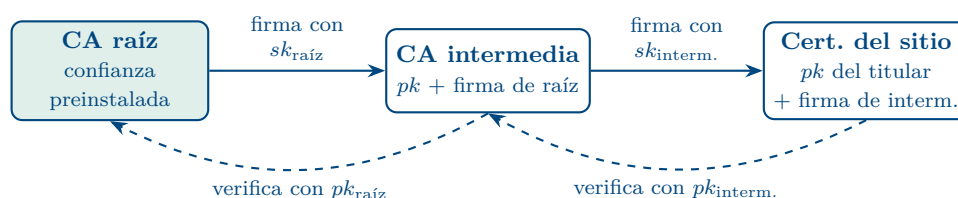
El cifrado asimétrico resuelve la confidencialidad pero *traslada* el problema: hay que distribuir la clave pública de forma **autenticada** (saber que la pública de Bob es realmente de Bob, si no un MiTM publica la suya). Esto lo resuelve la **PKI** (Public Key Infrastructure).

Definición: Certificado digital

Documento que asocia una **identidad** (nombre del titular) con su **clave pública**, y está **firmado** por una **Autoridad de Certificación (CA)** con la clave privada de la CA.

7.1. Cadena de confianza

Se confía en la CA raíz (cuya clave pública viene preinstalada en el SO/navegador). La CA raíz firma certificados de CAs intermedias, y estas a su vez firman certificados de titulares finales.



Cadena de certificados: cada nivel firma el certificado del siguiente con su clave privada. Validar = verificar las firmas hacia arriba hasta la CA raíz, cuya clave pública se conoce de antemano.

[Examen]

Certificados en parciales:

- **Validar un certificado = verificar la FIRMA de la CA** con la clave **pública de la CA**. (2018, 2017.)
- Un certificado contiene la clave **PÚBLICA del titular, no** la clave privada de la CA.
- **V/F “el certificado contiene la clave privada de la CA” → FALSO.** (2025-1, 2025-2.)
- La clave privada se mantiene **secreta**; la clave pública se distribuye de forma **autenticada** (para eso sirve el certificado). (2015, 2025-1.)

8. TLS handshake (resumen)

TLS combina todos los elementos vistos:

1. **Autenticación del servidor:** el servidor presenta su **certificado digital** (clave pública + firma de la CA). El cliente valida la cadena de certificados.
2. **Acuerdo de clave de sesión:** se ejecuta **ECDH** (Diffie-Hellman sobre curva elíptica) para derivar una clave de sesión efímera. No es posible descifrar tráfico pasado aunque se comprometa la clave privada del servidor (Forward Secrecy).
3. **Cifrado simétrico autenticado:** toda la comunicación posterior se cifra con **AES-GCM** usando la clave de sesión (confidencialidad + integridad en un solo paso).

9. Tabla comparativa: MAC vs Firma digital

Propiedad	MAC	Firma digital
Confidencialidad	No	No
Integridad	Sí	Sí
Autenticación	Sí (clave compartida)	Sí (clave privada)
No repudio	No	Sí
Verificación pública	No (necesita sk compartida)	Sí (basta pk)
Transferible	No	Sí
Clave requerida	Simétrica (compartida)	Par asimétrico

10. Ganchos mnemónicos

DH: “Alice sube, Bob baja, se encuentran en g^{xy} .” Alice envía g^x , Bob envía g^y ; cada uno eleva al exponente del otro. La clave no viaja: se *acuerda*.

RSA gen: “ $n = pq$, Fi de n , e coprimo, d inverso.” $(p-1)(q-1)$, $\gcd(e, \Phi) = 1$, $d = e^{-1} \text{ mód } \Phi$.

RSA operaciones: Cifro con la **pública** (m^e) = pública → cifrado. Descifro con la **privada** (c^d). Firmo con la **privada** ($H(m)^d$). Verifico con la **pública** (σ^e).

Certificado: “CA firma el DNI de Bob.” El DNI contiene la foto de Bob (= clave pública de Bob), no la tarjeta de crédito de la CA (= su privada).

MAC vs Firma: MAC es “acuerdo de socios” (solo entre los dos). Firma es “sello notarial” (cualquiera puede verificar, tiene validez legal).

11. Ejercicios tipo examen con respuesta

[Examen]

1. (2018, 2023) V/F: “Diffie-Hellman permite enviar de forma segura una clave a Bob.”

Respuesta: FALSO. DH **acuerda/deriva** una clave compartida en línea; no *transporta* una clave preexistente. Ambas partes derivan la misma clave g^{xy} sin que ninguno la haya elegido previamente ni enviado.

[Examen]

2. (2025-1, 2025-2) V/F: “Un certificado digital contiene la clave privada de la CA.”

Respuesta: FALSO. Un certificado contiene la **clave pública del titular** (e.g. del servidor web), firmada por la CA con la clave *privada* de la CA. Nunca contiene ninguna clave privada.

[Examen]

3. (Múltiples años) ¿Con qué clave se firma? ¿Con qué clave se verifica?

Respuesta: Se firma con la clave **privada** del emisor. Se verifica con la clave **pública** del emisor (inverso al cifrado). Confundirlos es error clásico de parcial.

[Examen]

4. (2025-2) ¿RSA-PKCS #1 v1.5 es CCA-Secure?

Respuesta: NO. El padding aleatorio lo hace CPA-Secure (el cifrado es no determinístico). Sin embargo, el cifrado RSA es **manipulable**; existen ataques de texto cifrado elegido (Bleichenbacher 1998). No provee integridad ni es cifrado autenticado. Para CCA-Secure se necesita OAEP + RSA, u otro esquema CCA-Secure.

[Examen]

5. (Recup 2025) ¿Por qué se elige q primo en DH?

Respuesta: En los grupos multiplicativos usados en DH, si q no fuera primo podría tener factores que generan subgrupos propios de orden menor. Trabajar con q primo garantiza que todos los elementos no nulos del grupo $\mathbb{Z}_q^*$ son coprimos entre sí, y no existen subgrupos “débiles” que el atacante pueda explotar para reducir el problema del logaritmo discreto.

[Examen]

6. (2017) Un proveedor de software quiere garantizar que sus binarios no fueron modificados. ¿Qué mecanismo usa y por qué?

Respuesta: Firma digital hashed RSA (o ECDSA). El proveedor firma $H(\text{binario})$ con su clave privada. Cualquier usuario verifica la firma con la clave pública del proveedor (públicamente verificable, sin compartir secretos). Esto garantiza integridad (el binario no fue modificado) y autenticación de origen.